

constexpr **INVOKE**

Contents

1. [Motivation](#)
2. [History](#)
3. [Proposal](#)
 - 3.1. [Wording](#)
4. [Acknowledgements](#)
5. [References](#)

§ 1. Motivation

Currently, one of the most important utility functions in the standard library, `std::invoke()`, is not `constexpr`. Even though `std::apply()` and `std::visit()`, both of which rely on `INVOKE`, are both `constexpr`. The standard library thus finds itself in an odd state where `std::invoke()` is and is not `constexpr`.

The reason that `std::invoke()` is not `constexpr` has some interesting history associated with it. But at this point, it is simply history, and there is no further blocker to making this change. This proposal resolves [LWG 2894](#) but also goes one step further and addresses various other `INVOKE`-related machinery.

§ 2. History

Our tale begins in April, 2015 with [llvm bug 23141](#), which presented this code which broke in clang in C++14 (but had compiled in C++11 mode) due to the introduction of a `constexpr __invoke` (which ended up breaking range-v3):

```
#include <functional>
#include <type_traits>

struct Fun
{
    template<typename T, typename U>
    void operator()(T && t, U && u) const
```

```

{
    static_assert(std::is_same<U, int &>::value, "");
}
};

int main()
{
    std::bind(Fun{}, std::placeholders::_1, 42)("hello");
}

```

as well as the similar [llvm bug 23135](#), which was about this program:

```

template<typename T>
int f(T x)
{
    return x.get();
}

template<typename T>
constexpr int g(T x)
{
    return x.get();
}

int main() {

    // O.K. The body of `f` is not required.
    decltype(f(0)) a;

    // Seems to instantiate the body of `g`
    // and results in an error.
    decltype(g(0)) b;

    return 0;
}

```

In both cases the fundamental issue was eager instantiation of the body, which doesn't actually seem necessary to determine the results here. In neither example is the return type deduced.

These are incarnations of [CWG 1581](#), which dealt with the question of when, exactly, are `constexpr` functions defined. In the broken programs above, the `constexpr` functions (the non-`const` call operator of the binder object being returned in the first case and `g()` in the second) were eagerly instantiated, triggering hard compile errors, in cases where the program ultimately would not have required their instantiation.

Thankfully, this difficult problem has been resolved by the adoption of [P0859](#) in Albuquerque, 2017. As a result, both of the above programs are valid.

This issue was the blocker for having a `constexpr std::invoke()` due to this eager instantiation issue - which no longer exists.

§ 3. Proposal

This proposal adds `constexpr` to the following `INVOKE`-related machinery: `invoke()`, `reference_wrapper<T>`, `not_fn()`, `bind()`, and `mem_fn()`. The remaining non-`constexpr` elements of the library that are `INVOKE`-adjacent are `function<Sig>`, `packaged_task<Sig>`, `async()`, `thread`, and `call_once()`.

The entirety of the wording is the addition of the `constexpr` keyword in 22 places.

§ 3.1. Wording

Add `constexpr` to several places in the synopsis in 19.14.1 [functional.syn]

```
namespace std {
    // [func.invoke], invoke
    template<class F, class... Args>
        constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)
            noexcept(is_nothrow_invocable_v<F, Args...>);

    // [refwrap], reference_wrapper
    template<class T> class reference_wrapper;

    template<class T> constexpr reference_wrapper<T> ref(T&) noexcept;
    template<class T> constexpr reference_wrapper<const T> cref(const T&) noexcept;
    template<class T> void ref(const T&&) = delete;
    template<class T> void cref(const T&&) = delete;

    template<class T> constexpr reference_wrapper<T> ref(reference_wrapper<T>) noexcept;
    template<class T> constexpr reference_wrapper<const T> cref(reference_wrapper<T>)

    // [arithmetic.operations], arithmetic operations
    // ...

    // [comparisons], comparisons
    // ...

    // [logical.operations], logical operations
    // ...

    // [bitwise.operations], bitwise operations
    // ...

    // [func.identity], identity
    // ...
```

```

// [func.not_fn], function template not_fn
template<class F> constexpr unspecified not_fn(F&& f);

// [func.bind], bind
template<class T> struct is_bind_expression;
template<class T> struct is_placeholder;

template<class F, class... BoundArgs>
    constexpr unspecified bind(F&&, BoundArgs&&...);
template<class R, class F, class... BoundArgs>
    constexpr unspecified bind(F&&, BoundArgs&&...);

namespace placeholders {
    // M is the implementation-defined number of placeholders
    see below _1;
    see below _2;
    .
    .
    .
    see below _M;
}

// [func.memfn], member function adaptors
template<class R, class T>
    constexpr unspecified mem_fn(R T::*) noexcept;

// ...
}

```

Add `constexpr` to the requirements of *forwarding call wrapper* in 19.4.3 [func.require]

Every call wrapper ([func.def]) shall be *Cpp17MoveConstructible*. A *forwarding call wrapper* is a call wrapper that can be called with an arbitrary argument list and delivers the arguments to the wrapped callable object as references. This forwarding step shall ensure that rvalue arguments are delivered as rvalue references and lvalue arguments are delivered as lvalue references. The defaulted move and copy constructor, respectively, of a forwarding call wrapper shall be a constexpr function if and only if all required element-wise initializations for copy and move, respectively, would satisfy the requirements for a constexpr function. The call operator of a forwarding call wrapper shall be a constexpr function if and only if the underlying call operation would satisfy the requirements of a constexpr function. A *simple call wrapper* is a forwarding call wrapper that is *Cpp17CopyConstructible* and *Cpp17CopyAssignable* and whose copy constructor, move constructor, copy assignment operator, and move assignment operator do not throw exceptions. [*Note:* In a typical implementation forwarding call wrappers have an overloaded function call operator of the form

```
template<class... UnBoundArgs>
constexpr R operator()(UnBoundArgs&&... unbound_args) cv-qual;
```

— end note]

Add `constexpr` to `std::invoke()` in 19.14.4 [func.invoke]

```
template<class F, class... Args>
constexpr invoke_result_t<F, Args...> invoke(F&& f, Args&&... args)
    noexcept(is_nothrow_invocable_v<F, Args...>);
```

Add `constexpr` to `std::reference_wrapper<T>` in 19.14.5 [refwrap]

```
namespace std {
    template<class T> class reference_wrapper {
    public:
        // types
        using type = T;

        // construct/copy/destroy
        template<class U>
        constexpr reference_wrapper(U&&) noexcept(see below );
        constexpr reference_wrapper(const reference_wrapper& x) noexcept;

        // assignment
        constexpr reference_wrapper& operator=(const reference_wrapper& x) noexcept;

        // access
        constexpr operator T& () const noexcept;
        constexpr T& get() const noexcept;

        // invocation
        template<class... ArgTypes>
        constexpr invoke_result_t<T&, ArgTypes...> operator()(ArgTypes&&...) const;
    };

    template<class T>
    reference_wrapper(T&) -> reference_wrapper<T>;
}
```

And its corresponding subsections, 19.14.5.1 [refwrap.const]

```
template<class U>
constexpr reference_wrapper(U&& u) noexcept(see below );
```

[...]

```
constexpr reference_wrapper(const reference_wrapper& x) noexcept;
```

19.14.5.2 [refwrap.assign]

```
constexpr reference_wrapper& operator=(const reference_wrapper& x) noexcept;
```

19.14.5.3 [refwrap.access]

```
constexpr operator T& () const noexcept;
```

[...]

```
constexpr T& get() const noexcept;
```

19.14.5.4 [refwrap.invoke]

```
template<class... ArgTypes>  
constexpr invoke_result_t<T&, ArgTypes...>  
operator()(ArgTypes&&... args) const;
```

and its helper functions, 19.14.5.5 [refwrap.helpers]

```
template<class T> constexpr reference_wrapper<T> ref(T& t) noexcept;
```

1 Returns: `reference_wrapper<T>(t)`.

```
template<class T> constexpr reference_wrapper<T> ref(reference_wrapper<T> t) noexcept
```

2 Returns: `ref(t.get())`.

```
template<class T> constexpr reference_wrapper<const T> cref(const T& t) noexcept;
```

3 Returns: `reference_wrapper<const T>(t)`.

```
template<class T> constexpr reference_wrapper<const T> cref(reference_wrapper<T> t)
```

4 Returns: `cref(t.get())`.

Add `constexpr` to `std::not_fn` in 19.14.11 [func.not_fn]

```
template<class F> constexpr unspecified not_fn(F&& f);
```

1 Effects: Equivalent to:

```
return call_wrapper(std::forward<F>(f));
```

where `call_wrapper` is an exposition only class defined as follows:

```
class call_wrapper {
    using FD = decay_t<F>;
    FD fd;

    explicit constexpr call_wrapper(F&& f);
public:
    constexpr call_wrapper(call_wrapper &&) = default;
    constexpr call_wrapper(const call_wrapper&) = default;

    template<class... Args>
        constexpr auto operator()(Args&&...) &
        -> decltype(!declval<invoke_result_t<FD&, Args...>>());

    template<class... Args>
        constexpr auto operator()(Args&&...) const&
        -> decltype(!declval<invoke_result_t<const FD&, Args...>>());

    template<class... Args>
        constexpr auto operator()(Args&&...) &&
        -> decltype(!declval<invoke_result_t<FD, Args...>>());

    template<class... Args>
        constexpr auto operator()(Args&&...) const&&
        -> decltype(!declval<invoke_result_t<const FD, Args...>>());
};
```

```
explicit constexpr call_wrapper(F&& f);
```

[...]

```
template<class... Args>
    constexpr auto operator()(Args&&...) &
    -> decltype(!declval<invoke_result_t<FD&, Args...>>());

template<class... Args>
    constexpr auto operator()(Args&&...) const&
    -> decltype(!declval<invoke_result_t<const FD&, Args...>>());
```

5 Effects: Equivalent to:

```
return !INVOKE(fd, std::forward<Args>(args)...); // see 19.14.3
```

```
template<class... Args>
constexpr auto operator()(Args&&...) &&
-> decltype(!declval<invoke_result_t<FD&, Args...>>());

template<class... Args>
constexpr auto operator()(Args&&...) const&&
-> decltype(!declval<invoke_result_t<const FD&, Args...>>());
```

6 Effects: Equivalent to:

```
return !INVOKE(std::move(fd), std::forward<Args>(args)...); // see 19.14.3
```

Add `constexpr` to `std::bind()` in 19.14.12.3 [func.bind.bind]

```
template<class F, class... BoundArgs>
constexpr unspecified bind(F&& f, BoundArgs&&... bound_args);
```

[...]

```
template<class R, class F, class... BoundArgs>
constexpr unspecified bind(F&& f, BoundArgs&&... bound_args);
```

Add `constexpr` to `std::mem_fn()` in 19.14.13 [func.memfn]

```
template<class R, class T> constexpr unspecified mem_fn(R T::* pm) noexcept;
```

§ 4. Acknowledgements

Thanks to Casey Carter and Agustín Bergé for going over the history of issues surrounding `constexpr` `invoke` and suggesting that this proposal be written. Thanks to Tomasz Kamiński and Tim Song for help on the wording.

§ 5. References

- [CWG1581] "When are constexpr member functions defined?" by Richard Smith, 2012-10-29
- [LWG2894] "The function template `std::apply()` is required to be constexpr, but `std::invoke()` isn't" by Great Britain, 2017-08-01

- [\[P0859\]](#) "Core Issue 1581: When are constexpr member functions defined?" by Richard Smith, 2017-11-09